

Synthèse mutualisée — Skills & MCP pour Riemann_Lab

[Synthèse mutualisée — Skills & MCP pour Riemann_Lab](#)

- 0. Méthode et fiabilité des sources
- 1. Tableau comparatif des 4 audits
- 2. Convergences fortes (confirmées par les 4 sources et par la vérification directe)
- 3. Points à trancher avant d'exécuter
- 3bis. Vérifications terrain — RAG BrainVault (27/09/2026)
- 3ter. Vérifications terrain — chiffres v15/v16 (27/09/2026)
- 4. Solution mutualisée recommandée
- 5. Plan de migration non destructif (fusion des vagues proposées par Kimi et DeepSeek)
- 6. Décision à prendre

Synthèse mutualisée — Skills & MCP pour Riemann_Lab

Point de repérage — croisement des 4 audits (Kimi, Perplexity, DeepSeek, Grok) + vérifications terrain RAG (27/09/2026)

0. Méthode et fiabilité des sources

Les quatre audits fournis dans le projet ont travaillé avec un accès très inégal au dépôt réel :

Source	Accès réel constaté	Fiabilité déclarée
Kimi	API GitHub complète (~350 fichiers, 4 skills lus en entier, site lu)	La plus fouillée — signale elle-même ses zones non lues (PDF, wiki, RAG)
Grok	Recherche web (40 puis 10 sources) — README + wiki + skills cités	Bonne synthèse mais pas de lecture ligne à ligne confirmée
Perplexity	35 sources, wiki via miroirs (github-wiki-see), pas de fetch direct de certaines pages	La plus prudente : signale elle-même 3 incohérences en fin de réponse
DeepSeek	Déclare explicitement auditer « par inférence » les .md/.pdf non lisibles via l'URL publique	La plus transparente sur ses limites, mais la moins vérifiée

Vérification effectuée en direct (API GitHub, branche `main` , 26/09/2026) pour trancher les points de désaccord :

Vérifié	Résultat
<code>.claude/skills/</code>	Exactement 4 skills : <code>phase-c-illinois</code> , <code>riemann-code-review</code> , <code>riemann-lab</code> , <code>riemann-security-review</code> — confirme Kimi et Grok, infirme le « 5 skills actifs » de Perplexity
Racine <code>main</code>	<code>.claude/</code> , <code>CLAUDE.md</code> , <code>CLAUDE_projet_zeta.md</code> , <code>Bonnes-Pratiques-Claude-Code.md</code> , <code>Guide-Git-GitHub.md</code> , <code>CONTRIBUTING.md</code> , <code>README.md</code> , <code>calculs/</code> , <code>config/</code> , <code>csv/</code> , <code>docs/</code> , <code>images/</code> , <code>logs/</code> , <code>pdf/</code> , <code>requirements/</code> , <code>scripts/</code> , <code>skills_export/</code> , <code>src/</code>
Absents confirmés	Pas de <code>tests/</code> , <code>.github/workflows/</code> , <code>AGENTS.md</code> , <code>.mcp.json</code> , <code>LICENSE</code> , <code>CHANGELOG.md</code> , <code>lean/</code> , <code>notebooks/</code> , <code>handoff/</code> versionné — confirme le constat unanime des 4 sources
Redondance <code>CLAUDE.md</code> / <code>CLAUDE_projet_zeta.md</code>	Confirmée — les deux fichiers coexistent toujours à la racine

Point tranché le 27/09/2026 (voir §3ter) : v15 (517 z/s) et v16 (1407 z/s, commit `00abe5c` du 08/08/2026) ne sont pas deux sources contradictoires — v16 remplace v15, x2,75 de gain sur le même benchmark T=100k. Ce n'était pas une incohérence des audits, juste une progression insuffisamment contextualisée. En revanche, une confusion plus importante a été détectée : ce chiffre de 1407 z/s est un **micro-benchmark**, pas le débit réel des grosses campagnes de calcul (~128 z/s mesuré sur un run réel de 21h38) — voir §3ter pour l'impact sur `research/roadmap.md` .

1. Tableau comparatif des 4 audits

Critère	Kimi	Perplexity	DeepSeek	Grok
Profondeur d'accès au repo	✔ Élevée (API complète)	🌀 Moyenne (miroirs wiki)	🌀 Faible (inférence déclarée)	🌀 Moyenne (web)
Transparence sur ses limites	✔ Explicite	✔ Explicite (auto-correction en fin de réponse)	✔ Explicite	⚠ Peu explicite
Exhaustivité de l'inventaire	Très élevée (A→E, 46 lignes)	Très élevée (structure + branches + wiki + site)	La plus exhaustive (46 items numérotés, priorisés)	Élevée (tableau unique + audit des 4 skills)
Valeur ajoutée spécifique	Seule à lire le contenu réel des 4 skills + repérer l'obsolescence de CLAUDE_projet_zeta.md (mai)	Contenu mathématique le plus riche (ξ(s), N(T) exact, fonctions L, formules explicites) + vigilance sur les incohérences	Rigueur d'audit (statuts ✔/⚠/✗), matrice priorité/impact, plan en 5 vagues détaillé	Seule à noter la culture « point d'arrêt avant run T>50k » comme acquis à préserver
Angle mort principal	Peu de détail sur le contenu mathématique pur	Moins structurée sur DevOps/CI/tests	Écoles internationales moins fines que DeepSeek n'aide pas ici — bien couvert en fait	Bibliographie internationale moins fournie que DeepSeek
Arborescence proposée	Additive, la plus prudente (migration en 2 temps, alias .bashrc)	Additive (research/ , rag/ , mcp/)	La plus complète (tests/ , docker/ , lean/ , .github/workflows/)	La plus minimale (5 dossiers)

Lecture croisée : les quatre s'accordent sur le diagnostic de fond (\$2). Elles se distinguent par la profondeur (Kimi et DeepSeek en tête sur l'exhaustivité), la prudence factuelle (Perplexity la plus fiable sur les chiffres), et la spécialisation (Perplexity = contenu mathématique fin, Kimi = audit du contenu réel des skills, DeepSeek = gouvernance/DevOps, Grok = synthèse pragmatique et culture projet).

2. Convergences fortes (confirmées par les 4 sources et par la vérification directe)

- Aucun protocole formel Claude Code ↔ Codex.** Ni AGENTS.md, ni .mcp.json, ni dossier handoff/ versionné — confirmé en direct.
- Aucun test automatisé ni CI.** Toute validation (Turing-Backlund, LMFDB) repose sur des runs manuels — confirmé (pas de tests/, pas de .github/workflows/).
- Étapes 2 (stats/GUE), 3 (IA locale/conjectures) et 4 (Lean 4) très en retard** sur l'étape 1 (calcul), qui est mature et documentée jusqu'à v15/v16.
- Aucune veille académique structurée** US / Chine / Russie, alors que c'est un objectif explicite du projet.
- Redondance** CLAUDE.md / CLAUDE_projet_zeta.md — confirmée en direct ; ce dernier est daté de mai et risque de désorienter un nouvel agent (Kimi le signale explicitement comme obsolète).
- Rôle ambigu de skills_export/ vs .claude/skills/** — les 4 sources demandent de clarifier lequel est la source de vérité.
- Un seul MCP mentionné** (.mcp.json GitHub), **non commité** — confirmé par Kimi qui l'a vu directement dans l'arbre.
- Le socle numérique (pipeline Illinois/Arb, cluster, skill phase-c-illinois) est le point le plus mature du projet** — aucune source ne propose d'y toucher, seulement de l'alléger (scinder règles vs journal d'optimisation).

3. Points à trancher avant d'exécuter

Point	Ce que disent les sources	Recommandation
Nombre de skills réellement actifs	Kimi/Grok : 4. Perplexity : 5 (non confirmé)	Tranché par vérification directe : 4 skills sur main
Chiffres v15 vs v16	Contradictoires selon les sources et pages consultées	Tranché par vérification terrain le 27/09/2026 — voir §3ter
Faut-il un skill Lean 4 dès maintenant ?	Toutes le proposent (lean-zeta , riemann-formal-lean , src/formal/lean/)	Créer un stub léger seulement (squelette Lakefile + fichiers vides), car l'étape 4 reste « long terme » dans ta propre roadmap
MCP « exotiques » (Wolfram, SageMath)	Proposés par DeepSeek/Grok	Priorité basse : ROI incertain vs coût de configuration sur ton matériel (8 Go RAM, i7-7500U)
Existence d'un Vault ZETA / RAG déjà en place	Kimi seul confirme scripts/rag_*.py (7 scripts) et /mnt/vault rag avec 838	Tranché par vérification terrain le 27/09/2026 — voir §3bis

chunks déjà indexés, site disant pourtant « RAG non opérationnel »

3bis. Vérifications terrain — RAG BrainVault (27/09/2026)

Après lecture des documents projet dédiés au RAG (`etat_rag_brainvault_20260704.md` , `Guide-RAG-BrainVault-Debutant.md` , `10_pave1-lexique_ia_brainvault_skills_rag_hprzeta.md` , `4_pave1-cerveau_autonome_zeta.md`) et vérification directe sur PC1 (SSD `vault_rag` rebranché) + sur GitHub, le point ouvert du §3 est **entièrement tranché**. Ces documents n’avaient été vus par aucun des 4 audits externes dans ce niveau de détail — ils changent la recommandation sur `zeta-vault` .

#	Vérification	Résultat
a	SSD <code>vault_rag</code> monté ?	Détecté comme <code>sdcl</code> (bon label, bon UUID) mais pas monté automatiquement — normal, branché après le démarrage. Monté manuellement avec succès (<code>sudo mount /mnt/vault_rag</code> , 228G libres sur 230G).
b	Chunks/collections réels	838 chunks , 1 seule collection <code>riemann_lab_corpus</code> — identique à l’ingestion du 05/07/2026, aucune dérive, aucun résidu de test.
c	Corpus à jour vs l’état actuel (v16, T=5M, sept. 2026) ?	Non . Dernier événement journalisé = une requête du 25/07/2026, pas une ingestion. Le corpus ignore tout depuis fin juillet.
d	Le « Brain Vault » Obsidian (concept <code>4_pave1-cerveau_autonome_zeta.md</code>) est-il construit ?	Non . Seulement 2 archives jamais dépliées (<code>brain-vault_ajout_primalite.zip</code> , <code>brain-vault_ajout_cryptozeta.zip</code>) dans <code>~/riemann_handoff/secrets_local/</code> . Le RAG technique (ChromaDB) est réel et opérationnel ; le Brain Vault éditorial (Obsidian) est resté au stade de zips.
e	Piège de discrimination de l’embedding (<code>all-MiniLM-L6-v2</code> qui confond tout, documenté dans le Guide §7) traité ?	Oui, corrigé le 25/07/2026 . <code>scripts/rag_query.py</code> fixe désormais <code>--k</code> par défaut à 8 (commentaire code explicite citant le symptôme et <code>Guide-Ollama-Pratique.md</code> §5.1). Explique la réponse test (IP du cluster) passée de 2/4 correctes le 19/07 à 4/4 le 25/07.
f	<code>scripts/rag_*.py</code> réellement committés (GitHub, branche <code>Riemann_Lab_IA</code>)	✓ <code>rag_query.py</code> , <code>rag_monitor.py</code> , <code>rag_ingest_corpus.py</code> tous présents. <code>scripts/inbox_ia_liens.md</code> étoffé à 130 lignes (34 au 04/07) — tranche déjà la question ouverte “code dupliqué sur <code>inbox-ia</code> ou indexé directement” : le code n’est jamais dupliqué, seul le Markdown est ingéré.
g	<code>.mcp.json</code> committé (n’importe quelle branche) ?	✗ Absent de <code>Riemann_Lab_IA</code> et de <code>main</code> — confirme qu’il n’a jamais été versionné.
h	Skill <code>zeta-vault</code> ou équivalent déjà ébauché dans <code>.claude/skills/</code> ?	✗ Absent. Le skill <code>riemann-lab</code> existant ne mentionne ni RAG, ni Vault — aucun chevauchement, feuille blanche. <code>AGENTS.md</code> également absent de cette branche.

Conséquence directe sur `zeta-vault` (§4.2) : ce skill n’est **pas** à créer from scratch — il existe déjà un pipeline RAG industrialisé, testé, et durci contre l’hallucination (`valeurs_non_ancrees()` , commit `01934df`). Le skill doit **documenter et exposer l’existant**, pas le reconstruire. Voir version corrigée en §4.2.

Chantier séparé identifié, hors périmètre skills actuel : la vision « Cerveau autonome zêta » (`4_pave1-cerveau_autonome_zeta.md`) propose deux agents (`hprzeta-knowledge-steward-agent` , `hprzeta-disaster-recovery-agent`), un vault Obsidian versionné Git, Restic + Syncthing, un Raspberry Pi/VPS observateur (stratégie 3-2-1). Rien de tout cela n’est commencé. C’est un projet à part entière, à trancher séparément — il ne doit pas être mélangé aux skills RAG construits maintenant.

3ter. Vérifications terrain — chiffres v15/v16 (27/09/2026)

Croisement de `docs/index.html` (site public), `Riemann_Lab.wiki/STACK.md` (référence stable) et `Riemann_Lab.wiki/JOURNAL.md` (journal daté), sur demande explicite de vérifier la divergence signalée par Perplexity avant de figer `research/roadmap.md` .

Ce n’était pas une incohérence — c’est une progression mal contextualisée :

	v15	v16
Benchmark T=100k (site + STACK.md)	517 z/s, 4,4 min	1407 z/s, 1,6 min (×2,75)
Commit	—	00abe5c , 08/08/2026
Validation	—	138 069 zéros, 0 manquant, Turing complet, LMFDB 20/20

v16 remplace v15 sur le même test de vitesse ; ce n’est pas deux pipelines qui coexistent avec des chiffres contradictoires sur la même chose.

Ce qui est réellement important, et que ni les 4 audits ni la vérification GitHub initiale n’avaient identifié : le chiffre 1407 z/s (comme le 517 z/s de v15) est un **micro-benchmark** sur une plage courte (T=100k), pas le débit réel des grosses campagnes de calcul. Sur les vrais runs de production, JOURNAL.md mesure :

Date	Run	Résultat	Débit réel
06/08/2026	v15, plage [14, 3 469 743] (T≈3,47M)	6 749 092 zéros	—
18/08/2026	v16, run principal (21h38)	10 016 297 / 10 016 474 attendus (déficit 177)	128,56 z/s

Impact concret pour research/roadmap.md : toute estimation de capacité doit se baser sur ~128 z/s (débit réel constaté sur un run long), pas sur 1407 z/s (benchmark T=100k) — sinon l’estimation est fausse d’un facteur ×11.

Anomalie non résolue, découverte au passage — à trancher avec toi, pas par moi : - Test A/B du 18/08 puis diagnostic du 05/09 : MARGE=10.0 (grille 5× plus fine) trouve **63 zéros de moins** que MARGE=2.0 sur la même plage — « contredit la monotonie ». Toujours non résolu à la dernière entrée lue avant le 13/09 (qui traite d’un sujet voisin, MARGE_SECURITE , sans confirmer que l’anomalie des 63 zéros est close). - Deux chiffres différents circulent pour « le » run T≈5M : **10 016 377** (mémoire du 04/07/2026) vs **10 016 297/474** (JOURNAL.md du 18/08/2026) — deux runs distincts à des dates différentes, mais lequel fait référence aujourd’hui n’est pas déterminable depuis les documents seuls.

Ces deux points sont des questions de fond sur la validité numérique du pipeline, pas de simples vérifications de chiffres — ils reviennent à toi pour trancher, avant de citer un débit ou un total de zéros dans research/roadmap.md .

4. Solution mutualisée recommandée

4.1 Skills existants à mettre à jour (ne rien casser)

Skill	Action	Justification (source)
riemann-code-review	Intégrer les leçons v12→v16 (Arb, SEUIL_1NEWTON , cache RS, précision fixe 64 bits)	Daté du 1er juin 2026, obsolète (Grok, Kimi)
riemann-security-review	Ajouter une section MCP (tokens transportés par les serveurs) + secrets dans logs/CSV	Daté du 1er juin 2026 (Grok, Kimi)
phase-c-illinois	Scinder en « règles permanentes » (courtes, toujours chargées) + « journal d’optimisation » (historique v4→v16, chargé à la demande)	~480 lignes, risque de dilution de contexte (Grok) ; cohérent avec tes propres seuils de contexte 50/70/80 %
riemann-lab	Alléger la partie calcul (renvoyer vers phase-c-illinois), externaliser l’historique v12→v16 vers le wiki	Deviens un « fourre-tout » (Grok, Kimi)
CLAUDE_projet_zeta.md	Fusionner dans CLAUDE.md , archiver l’ancienne version (jamais supprimer)	Obsolescence confirmée en direct + par Kimi/DeepSeek

4.2 Skills à créer (dédupliqués entre les 4 sources, noms harmonisés)

#	Skill	Rôle	Priorité	Convergence
1	riemann-agent-bridge (≈ AGENTS.md + protocole handoff)	Contrat commun Claude Code ↔ Codex ↔ Ollama, format de passation handoff/claude_to_codex.md / codex_to_claude.md	● Haute	4/4
2	riemann-literature-scout (= riemann-litwatch chez Kimi, arxiv-veille-zeta chez	Veille arXiv + universités US (Princeton/IAS, MIT, Michigan, Berkeley, Stanford) / CN (Tsinghua, Peking, CAS) / RU (Steklov, MGU) ; résumé gradué français, cohérent	● Haute	4/4

	DeepSeek)	avec ta pédagogie		
3	riemann-statistics-gue	Écart normalisés, corrélation de paires de Montgomery, comparaison GUE vs Poisson (Étape 2 roadmap)	● Haute	4/4
4	riemann-lmfdb-cross-validation	Export CSV local → requête LMfdb → diff → alerte si écart	● Moyenne	3/4 (déjà partiellement couvert par turing_validation.py existant)
5	riemann-formal-lean (stub)	Squelette Lean 4/Mathlib : prolongement analytique, équation fonctionnelle $\xi(s)=\xi(1-s)$	● Utile, non urgente	4/4, mais roadmap = long terme
6	riemann-ai-conjecture	Ollama/Mathstral comme premier filtre local avant appel Claude/Codex ; exploration de conjectures sur les CSV de zéros	● Utile	3/4
7	zeta-vault (wrapper de l'existant — voir §3bis, ne pas recréer)	Documente et expose rag_query.py --k 8 / rag_monitor.py / rag_ingest_corpus.py à Claude Code/Codex ; réflexe obligatoire : mountpoint /mnt/vault_rag avant toute requête (SSD hot-plug, pas de montage auto) ; déclenche une ré-ingestion (corpus figé depuis le 25/07/2026, ~2 mois de retard : v16, T=5M, sessions skills manquants)	● Haute (upgradée)	4/4 dans l'esprit, confirmé par vérification terrain
8	riemann-proof-status (nouveau, ajouté le 27/09/2026)	Registre append-only à 5 statuts honnêtes (PROVED , CONJECTURED , VERIFIED_NUMERICALLY , HEURISTIC , HANDED_OFF) pour toute affirmation mathématique du projet — formalise l'exigence déjà en place « distinguer prouvé/conjecturé/heuristique ». Inspiré de evomath-tao (dépôt EvoScientist/EvoSkills , Apache 2.0, 436★) et de research-state (plugin mathbox de nidrissi , MIT) — reformulé sur mesure, aucune dépendance externe importée. Détecte aussi la fraîcheur (résultat CONJECTURED / HANDED_OFF non revisité depuis >2 mois)	● Utile, non urgente	Candidat externe croisé avec 2 sources, non un point des 4 audits d'origine

4.2bis Sources externes évaluées le 27/09/2026 — non retenues telles quelles

Source	Nature	Verdict
Plugin mathbox (nidrissi , MIT, v3.x)	10 skills de gestion de programme de recherche (research-program , proof-audit , computation-audit , manuscript-integrate ...)	Pas un système de calcul formel — gère le processus de recherche, pas les calculs eux-mêmes. research-state a inspiré riemann-proof-status (§4.2 item 8) plutôt que d'être importé en bloc, pour éviter le recouvrement avec riemann-agent-bridge déjà en place.
Dépôt EvoScientist/EvoSkills (Apache 2.0, 436★, 17 skills)	Pack pour agent de recherche scientifique généraliste (ML expérimental, rédaction de papiers, slides)	16 des 17 skills sont hors sujet (pipeline ML, rebuttal de papier, slides académiques) — seul evomath-tao (protocole de preuve à 5 statuts) est transférable, et a inspiré riemann-proof-status de la même façon. Ne pas importer le pack entier : bruit + dépendances externes (Semantic Scholar, HuggingFace, Gemini) non auditées.

⚠ Sources vérifiées via des pages tierces (claudepluginhub.com, GitHub) le 27/09/2026, pas la documentation officielle Anthropic — à revérifier avant toute intégration réelle (numéro de version mathbox notamment : capture d'écran à 3.1.0, page web à 3.2.0, écart non expliqué).

4.3 MCP à intégrer, par priorité

Priorité	Serveur MCP	Usage	Remarque matériel (8 Go RAM)
----------	-------------	-------	------------------------------

● 1	GitHub (déjà évoqué dans un <code>.mcp.json</code> non commité)	Issues, PR, Kanban, wiki	Léger — juste à committer proprement
● 2	<code>filesystem</code>	Accès repo + <code>/mnt/data</code> + <code>/mnt/vault_rag</code>	Léger
● 3	<code>arxiv-mcp-server</code>	Alimente <code>riemann-literature-scout</code>	Léger (réseau uniquement)
● 4	<code>sqlite</code> (sur un futur <code>data/zeros.db</code>)	Requêtes indexées sur les 10M zéros au lieu de CSV bruts	À créer — actuellement absent
● 5	<code>lean</code> (custom)	Compilation Lean 4, LSP mathlib4	À différer tant que le skill Lean reste un stub
● 6	<code>sympy-mcp</code>	Calcul symbolique fiable (évite les hallucinations sur les formules)	Léger
● Optionnel	<code>wolfram</code> , <code>sagemath</code>	Vérifications croisées ponctuelles	Coût de config élevé pour gain marginal — à ne considérer qu'en dernier

4.4 Protocole Claude Code ↔ Codex ↔ Ollama (synthèse des 4 propositions)

- **Claude Code** : architecture, refactoring, rédaction mathématique/Lean, orchestration, gardien des conventions (`CLAUDE.md`).
- **Codex** : génération de code algorithmique expérimental, tests unitaires, scripts `C/arb_C`, exécution sandbox.
- **Ollama local (Mathstral)** : premier filtre / conjectures exploratoires offline, avant sollicitation des agents distants (économie de tokens — DeepSeek).
- **Passation** : `handoff/claude_to_codex.md` et `handoff/codex_to_claude.md`, sur le modèle du « mécanisme point projet » déjà présent dans ton `CLAUDE.md` — une extension naturelle de l'existant, pas une nouveauté.
- **Point d'arrêt obligatoire** avant tout run `T > 50k` (Grok) — cohérent avec ta culture de validation déjà en place.

4.5 Arborescence additive finale (fusion des 4 propositions, aucune suppression)

Principe unanime : **on n'ajoute que là où c'est vide** ; `src/calculs/optimisation/`, `docs/`, `scripts/`, `.claude/skills/` existants restent intouchés. Kimi ajoute une contrainte concrète que les autres n'ont pas : tes alias `~/bashrc` (`zeta-run`, `zeta-progress`, `zeta-distribute`) pointent vers `scripts/` à la racine — tout déplacement de scripts doit passer par des liens symboliques temporaires avant mise à jour du `.bashrc`.

```
Riemann Lab/
├── AGENTS.md                # racine – inchangée
├── CLAUDE.md                # NOUVEAU – contrat Claude Code ↔ Codex ↔ Ollama
├── CLAUDE_projet_zeta.md    # INCHANGÉ (fusion à faire en dernier, §5)
├── CHANGELOG.md            # → À FUSIONNER dans CLAUDE.md, archivé ensuite
├── .mcp.json               # NOUVEAU (léger, pas urgent)
├──                          # NOUVEAU – serveurs MCP du §4.3
├──
├── .claude/skills/         # INCHANGÉ + skills du §4.2
├──
├── handoff/                # NOUVEAU (versionné, contrairement au Handoff.md local actuel)
│   ├── claude_to_codex.md
│   └── codex_to_claude.md
├──
├── research/               # NOUVEAU – cœur recherche maths
│   ├── roadmap.md          # objectifs 3/6/12 mois
│   ├── bibliographie/      # US / Chine / Russie, alimenté par riemann-literature-scout
│   └── analysis/           # gaps_between_zeros.py, explicit_formulas.py, xi_function.py
├──
├── src/                    # NOUVEAU – cœur v2-v16
│   ├── calculs/optimisation/ # INCHANGÉ – cœur v2-v16
│   ├── stats/              # NOUVEAU – Étape 2 (Montgomery/GUE)
│   ├── ia/                 # existant, à enrichir (rag/, conjecture-mining)
│   └── formal/lean/         # NOUVEAU – stub uniquement
├──
├── tests/                  # NOUVEAU – pytest (illinois, scan_arb, turing)
├── .github/workflows/      # NOUVEAU – CI légère (compile C, tests rapides T=100)
├──
├── docs/, pdf/, scripts/, config/, requirements/, csv/, logs/, images/, calculs/, skills_export/ #
└── INCHANGÉS
```

Options différées (proposées par DeepSeek/Perplexity, non retenues en premier jet car hors du périmètre explicite de tes instructions) : `docker/`, `notebooks/`, `benchmarks/`, `LICENSE`, `CITATION.cff`, `.zenodo.json`. À reconsidérer une fois le socle skills/MCP/tests en place.

5. Plan de migration non destructif (fusion des vagues proposées par Kimi

Vague	Contenu	Risque
V1	AGENTS.md , .mcp.json.example (GitHub + filesystem ; arxiv différé — voir note), handoff/	✔ Fait le 27/09/2026 — commit df46f82 sur Riemann_Lab_IA (5224262..df46f82)
V2	✔ Fait le 27/09/2026 — commit 9e0b203 . Créer les 3 skills prioritaires (riemann-agent-bridge , riemann-literature-scout , riemann-statistics-gue)	⦿ Nul
V3	✔ Fait le 27/09/2026 — commits 5e3a44d puis 70e7463 (correctif). Mettre à jour riemann-code-review et riemann-security-review (contenu, pas de suppression)	⦿ Nul
V4	tests/ + .github/workflows/ (CI légère)	⦿ Faible — nécessite de figer les entrées de illinois_refine , scan_arb
V5	Scinder phase-c-illinois (règles vs journal) + fusionner CLAUDE_projet_zeta.md dans CLAUDE.md + migrer les scripts avec liens symboliques temporaires (contrainte .bashrc de Kimi)	⦿ Moyen — à faire en dernier, avec confirmation explicite avant commit

Note V1 — écart par rapport au plan initial : .mcp.json réel est resté **local et ignoré par git** (.gitignore:40) — c’est une règle de sécurité déjà en place avant cette synthèse (probablement liée à un incident de token, mentionné dans l’audit sécurité), pas un oubli. On a donc committé un gabarit .mcp.json.example (avec \${GITHUB_TOKEN} en variable d’environnement, jamais de secret en clair) plutôt que de forcer le fichier réel dans git. Le serveur MCP arxiv n’a pas été activé faute de paquet vérifié — à choisir et tester séparément avant de l’ajouter au gabarit.

6. Décision à prendre

Cette synthèse recoupe les quatre audits, résout leurs 2 divergences factuelles par une vérification directe du dépôt, et propose une solution unique mutualisant leurs points forts respectifs (rigueur de Kimi et DeepSeek, richesse mathématique de Perplexity, pragmatisme de Grok). Avant de lancer l’exécution, trois choix te reviennent :

- Ordre d’exécution — Fait (27/09/2026) :** Vagues V1 (df46f82), V2 (9e0b203) et V3 (5e3a44d) poussées sur Riemann_Lab_IA — AGENTS.md , .mcp.json.example , handoff/ , les 3 skills prioritaires (riemann-agent-bridge , riemann-literature-scout , riemann-statistics-gue), et mise à jour de riemann-code-review / riemann-security-review (leçons v12→v16, sécurité MCP). Reste : Vagues V4-V5 (tests/CI, scission phase-c-illinois , fusion CLAUDE_projet_zeta.md).
- ~~zeta-vault~~ /RAG existant — **Tranché (§3bis, 27/09/2026) :** le RAG existe, est opérationnel, et le skill devient un wrapper de l’existant + déclencheur de ré-ingestion.
- Vérification des chiffres v15/v16 — Tranché (§3ter, 27/09/2026) :** v16 remplace v15 (×2,75), pas d’incohérence. Mais le 1407 z/s est un micro-benchmark T=100k — research/roadmap.md doit utiliser le débit réel de production (~128 z/s), sous peine d’estimation fausse d’un facteur ×11.
- Nouveau** — Ré-ingestion RAG : je lance rag_ingest_corpus.py maintenant pour rattraper les 2 mois de retard, ou on l’intègre proprement comme première tâche du futur skill zeta-vault (Vague V2) ?
- Nouveau** — Vision « Cerveau autonome zêta » (agents Knowledge Steward/Disaster Recovery, Obsidian, Raspberry Pi) : hors périmètre pour l’instant, ou tu veux l’ouvrir comme chantier séparé ?
- Nouveau (§3ter) —** Anomalie des 63 zéros (MARGE=10.0 vs MARGE=2.0 , contredit la monotonie, non résolue au 05/09) et ambiguïté sur le run T~5M de référence (10 016 377 vs 10 016 297/474) : à trancher avec toi avant de citer un total de zéros dans research/roadmap.md — question numérique de fond, pas une simple vérification.
- Nouveau (découvert le 27/09/2026, hygiène de session) —** La section PROMPT_REPRISE de Handoff.md devait être **écrasée** à chaque fin de session (règle déjà écrite dans le fichier lui-même) mais ne l’a pas été depuis plusieurs sessions : elle avait accumulé tout l’historique depuis le 06/09/2026 (>1000 lignes, ~86 Ko) au lieu de ne garder que l’état courant. Correction demandée dans le prompt de reprise du 27/09 : migrer vers JOURNAL.md tout ce qui n’y est pas déjà, puis remplacer (pas ajouter) le bloc par l’état courant seul. À vérifier que ça tient dans la durée (pas de régression aux prochaines sessions).

